ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 10

(№4. Թվային հաջորդականության տատանումներ)

Աշխատանքի նպատակ

Ստեղծել դատարկ և տվյալներով օբյեկտներ, ապահով փոփոխել դինամիկ զանգվածի չափը:

Խնդրի պահանջ Սահմանել `SequenceStudy` դասը, որը պահպանում է `a` թվերի դինամիկ զանգվածը և `p` շեմային արժեքը: Հարևան տարրերի տարբերության մոդուլը սահմանվում է հետևյալ բանաձևով.

d[i] = |a[i+1] - a[i]|
|---|

1. Հաշվարկային պահանջներ

- Հաշվել բոլոր d[i]-երի գումարը:
- Գտնել ամենամեծ d[i]-ի ձախ տարրի ինդեքսը:
- Հաշվել p-ից խիստ մեծ տատանումների (d[i] > p) ընդհանուր քանակը:

2. Ընդհանուր պահանջներ

- **Դասի դաշտեր:** `private double* data`, `int n` (զանգվածի չափ), `double p` (շեմային արժեք):
- **Լռելյայն կոնստրուկտոր:** Ստեղծում է դատարկ վիճակով օբյեկտ (`n = 0`, `data = nullptr`, `p = 0`):
- **Մեթոդներ:** `append(value)`, `removeAt(index)`, `print()`, ինչպես նաև 3 հաշվարկային `const` մեթոդներ:
- **Սահմանափակումներ:** Սկզբնական ոչ դատարկ չափը պետք է բավարարի 3 <= n <= 30 պայմանին: Էլեմենտների ավելացման կամ հեռացման ընթացքում չափը կարող է փոփոխվել 0 <= n <= 30 տիրույթում:

- **Սահմանային դեպքեր:** Դատարկ կամ ոչ բավարար տարրեր ունեցող վիճակում (n < 2) գումարը և քանակը ընդունում են 0 արժեքը, իսկ ինդեքսը՝ -1 («Տվյալները բավարար չեն» հաղորդագրությամբ):
- **Հիշողության կառավարում:** Օբյեկտների բացահայտ պատճենում կամ վերագրում չի նախատեսվում:

Տեսական մաս

- **Լռելյայն կոնստրուկտոր (Default Constructor):** Կանչվում է առանց արգումենտների կամ բոլոր պարամետրերի համար լռելյայն արժեքներ ունենալու դեպքում: Դատարկ օբյեկտում `n = 0` և `data = nullptr` արժեքների առկայությունը համարվում է լիարժեք վավեր վիճակ և կանխում է չնախատեսված վայրի ցուցիչների (wild pointers) առաջացումը:
- **Դինամիկ հիշողության անվտանգ փոփոխում:** Զանգվածի տարր ավելացնելիս կամ հեռացնելիս նախ հատկացվում է նոր հիշողության բլոկ (`new double[new_n]`), առկա տվյալները պատճենվում են նոր տիրույթ, որից հետո միայն ազատվում է նախորդ հիշողությունը (`delete[] data`): Ցուցիչը վերահասցեավորվում է նոր բլոկի վրա, ինչը բացառում է հիշողության արտահոսքը (memory leak):

C++ Ծրագրի կոդը

```
#include <iostream>

#include <cmath>

#include <iomanip>

using namespace std;

class SequenceStudy {

private:

    double* data;

    int n;
```

```
    double p;
    const double EPS = 1e-9;
public:
    SequenceStudy(double threshold = 0.0) {
        data = nullptr;
        n = 0;
        p = (threshold >= 0) ? threshold : 0.0;
    }
    SequenceStudy(const double* arr, int size, double threshold) {
        data = nullptr;
        n = 0;
        p = (threshold >= 0) ? threshold : 0.0;
        if (size >= 3 && size <= 30 && arr != nullptr) {
            data = new double[size];
            n = size;
            for (int i = 0; i < size; ++i) {
                data[i] = arr[i];
            }
```

```
    } else {
        cout << "[Սխալ] Սկզբնական չափը պետք է լինի 3-ից 30 միջակայքում:\n";
    }
}
~SequenceStudy() {
    delete[] data;
    data = nullptr;
}
bool append(double value) {
    if (n >= 30) {
        cout << "[Սխալ] Զանգվածի առավելագույն չափը (30) լրացված է:\n";
        return false;
    }

    double* newData = new double[n + 1];
    for (int i = 0; i < n; ++i) {
        newData[i] = data[i];
    }
    newData[n] = value;
```

```
    delete[] data;
    data = newData;
    n++;
    return true;
}
bool removeAt(int index) {
    if (index < 0 || index >= n) {
        cout << "[Սխալ] Անվավեր ինդեքս։\n";
        return false;
    }
    if (n == 1) {
        delete[] data;
        data = nullptr;
        n = 0;
        return true;
    }
    double* newData = new double[n - 1];
    for (int i = 0, j = 0; i < n; ++i) {
```

```
        if (i != index) {
            newData[j++] = data[i];
        }
    }
    delete[] data;
    data = newData;
    n--;
    return true;
}
double calculateSumD() const {
    if (n < 2) {
        return 0.0;
    }
    double sum = 0.0;
    for (int i = 0; i < n - 1; ++i) {
        sum += fabs(data[i + 1] - data[i]);
    }
    return sum;
}
```

```
int findMaxDIndex() const {
    if (n < 2) {
        cout << "Տվյալները բավարար չեն\n";
        return -1;
    }
    int maxIdx = 0;
    double maxD = fabs(data[1] - data[0]);
    for (int i = 1; i < n - 1; ++i) {
        double currentD = fabs(data[i + 1] - data[i]);
        if (currentD > maxD + EPS) {
            maxD = currentD;
            maxIdx = i;
        }
    }
    return maxIdx;
}
int countGreaterD() const {
    if (n < 2) {
```

```
            return 0;
        }
        int count = 0;
        for (int i = 0; i < n - 1; ++i) {
            double d = fabs(data[i + 1] - data[i]);
            if (d > p + EPS) {
                count++;
            }
        }
        return count;
    }
    void print() const {
        cout << "Զանգված (n = " << n << ", p = " << p << "): [ ";
        for (int i = 0; i < n; ++i) {
            cout << data[i] << (i < n - 1 ? ", " : " ");
        }
        cout << "]\n";
    }
};
```

```
int main() {
    setlocale(LC_ALL, "Armenian");
    cout << "=== ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 10 (Վարիանտ №4) ===\n\n";
    cout << "--- Օբյեկտ 1 (Դատարկ օբյեկտ append-ով) ---\n";
    SequenceStudy obj1(0.0);
    double initial_data[] = {2, 5, 4, 10};
    for (double val : initial_data) {
        obj1.append(val);
    }
    obj1.print();
    cout << "d[i]-երի գումարը: " << obj1.calculateSumD() << endl;
    cout << "Ամենամեծ d[i]-ի ձախ ինդեքսը: " << obj1.findMaxDIndex() << endl;
    cout << "p (0)-ից մեծ փոփոխությունների քանակը: " << obj1.countGreaterD() <<
endl;
    cout << "\n-> Հեռացնում ենք ինդեքս 1-ի տարրը...\n";
    obj1.removeAt(1);
    obj1.print();
    cout << "Նոր d[i]-երի գումարը: " << obj1.calculateSumD() << endl;
    cout << "\n--- Օբյեկտ 2 (Ստուգիչ օրինակ p=2-ով) ---\n";
```

```
    double a[] = {2, 5, 4, 10};

    SequenceStudy obj2(a, 4, 2.0);

    obj2.print();

    cout << "d[i]-երի գումարը (Ակնկալվող՝ 10): " << obj2.calculateSumD() << endl;

    cout << "Ամենամեծ d[i]-ի ձախ ինդեքսը (Ակնկալվող՝ 2): " << obj2.findMaxDIndex()
<< endl;

    cout << "p (2)-ից մեծ փոփոխությունների քանակը (Ակնկալվող՝ 2): " <<
obj2.countGreaterD() << endl;

    return 0;

}
```